

GitHub, 17 Aug 2026

How retries amplified the overload

Every architecture diagram in this episode is available under CC BY 4.0.

Primary sources: githubstatus.com incident zkxwbgr0cnmx (RCA and updates); GitHub availability report, August 2026 (github.blog); GitHub CTO blog post, 20 Aug 2026.

All times UTC, as GitHub reports them, 12-hour (ART = UTC−3).

The lesson: an Istio sidecar hit its concurrency limit while autoscaling watched the host service; retries in two layers (gateway and a client bug) multiplied load on the shared auth path. GitHub recovered by refusing some requests on purpose and ramping back site by site.

Contents

2. UML sequence (simplified): two retry layers on one saturated load balancer
3. Microservice topology (simplified; GitHub does not publish it): many products, one shared gateway auth path
4. Retry layers multiply: $3 \times 3 = 9$ calls per click (illustrative)
5. Timeline, Aug 17, 2026 (UTC, 12-hour)
6. Failure dynamics: first event, signal, shared resource, containment, trade-off, recovery
7. Aug 6, 2026: the self-amplifying Actions backlog
8. Monday: six recommendations (our opinion), each with what it does NOT solve

UML sequence (simplified): two retry layers on one saturated load balancer

[src/seq_retry_two_layers.d2](#) · [svg/seq_retry_two_layers.svg](#) · [png/seq_retry_two_layers.png](#)

UML sequence (simplified): two retry layers, Aug 17, 2026



Worst case: client $3 \times$ gateway $3 = 9$ calls per click (illustrative).

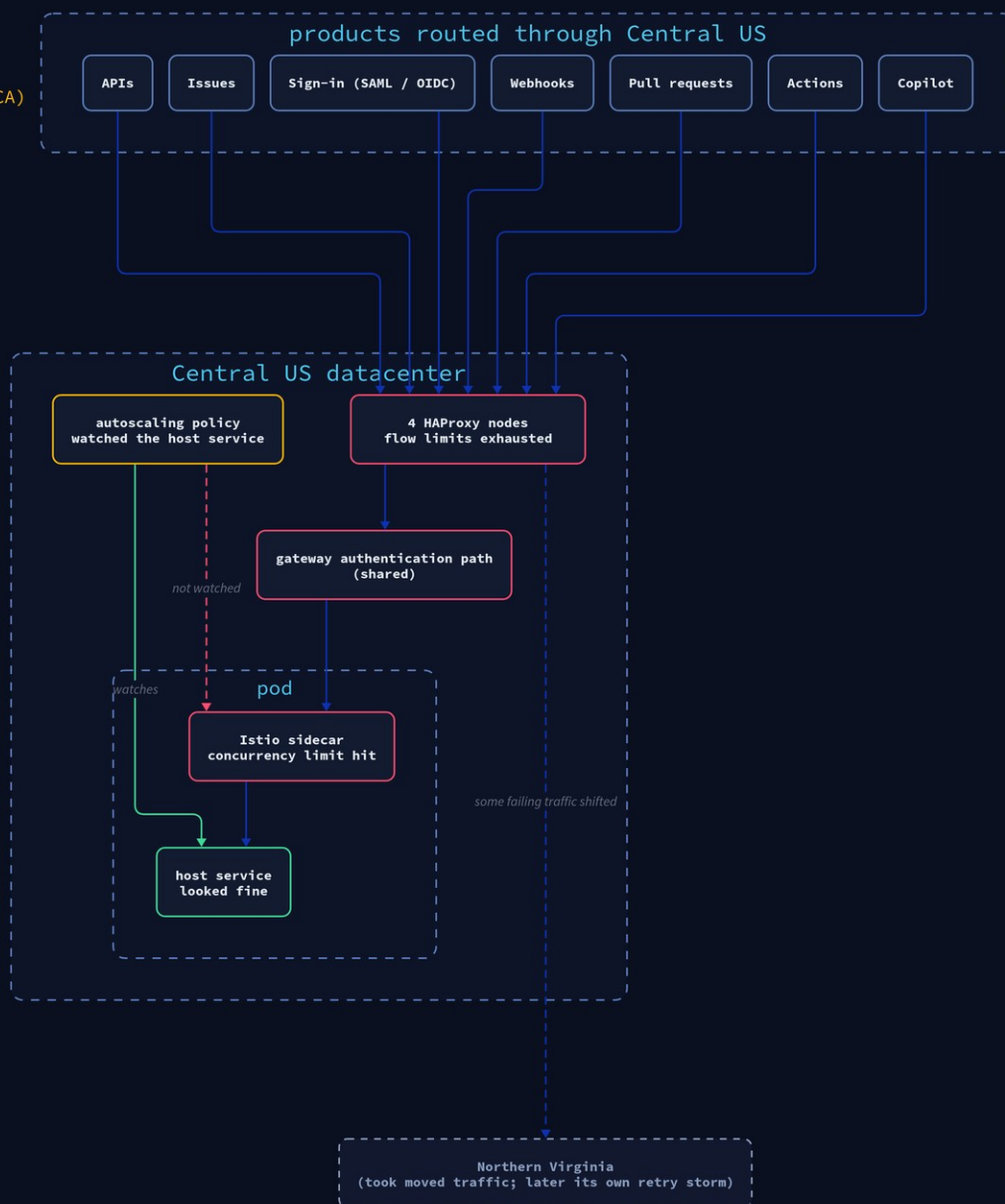
GitHub's real figure: Copilot Token Service 7–9K $\rightarrow$ 70–100K requests/s (about $10\times$).

Sources: [githubstatus.com](#) incident RCA; GitHub availability report (Aug 2026). CC BY 4.0 · Systems, Drawn

Microservice topology (simplified; GitHub does not publish it): many products, one shared gateway auth path

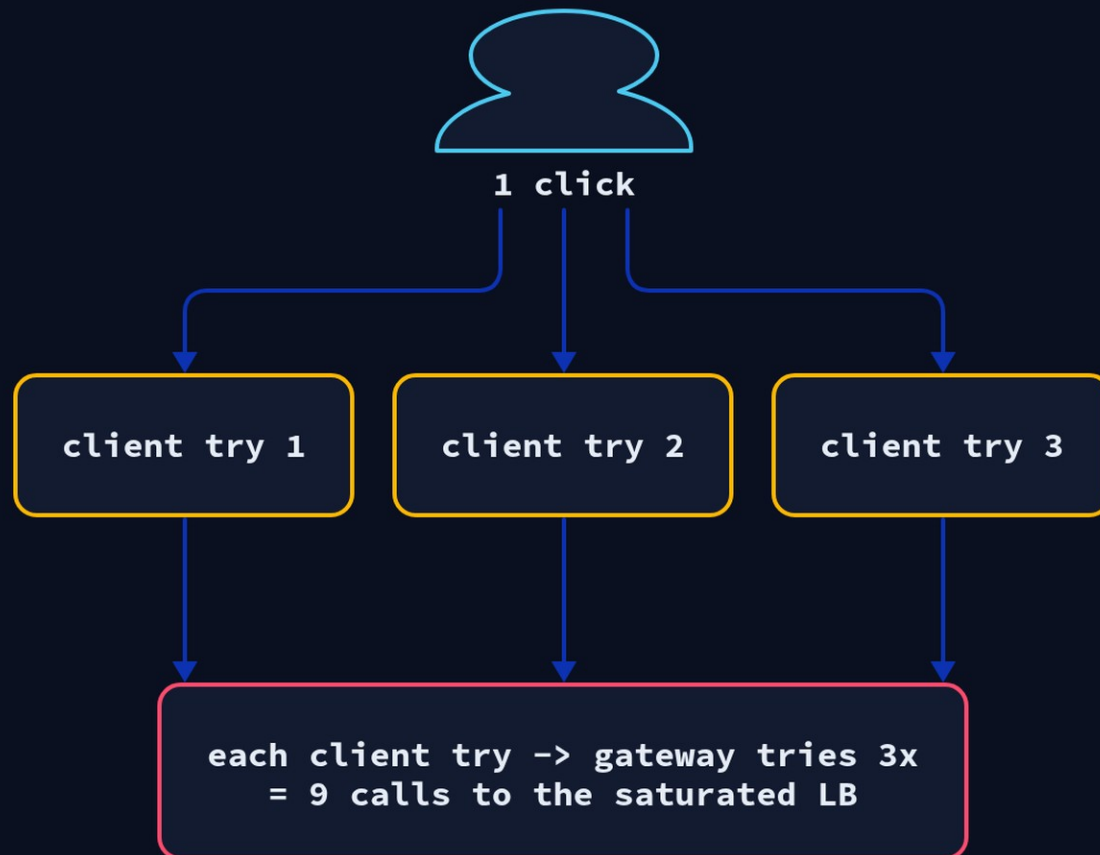
src/topology_shared_auth.d2 · svg/topology_shared_auth.svg · png/topology_shared_auth.png

simplified · GitHub doesn't publish this topology
(hop order, fan-in and node count are our drawing from the RCA)



Retry layers multiply: $3 \times 3 = 9$ calls per click (illustrative)

[src/retry_multiplier.d2](#) · [svg/retry_multiplier.svg](#) · [png/retry_multiplier.png](#)

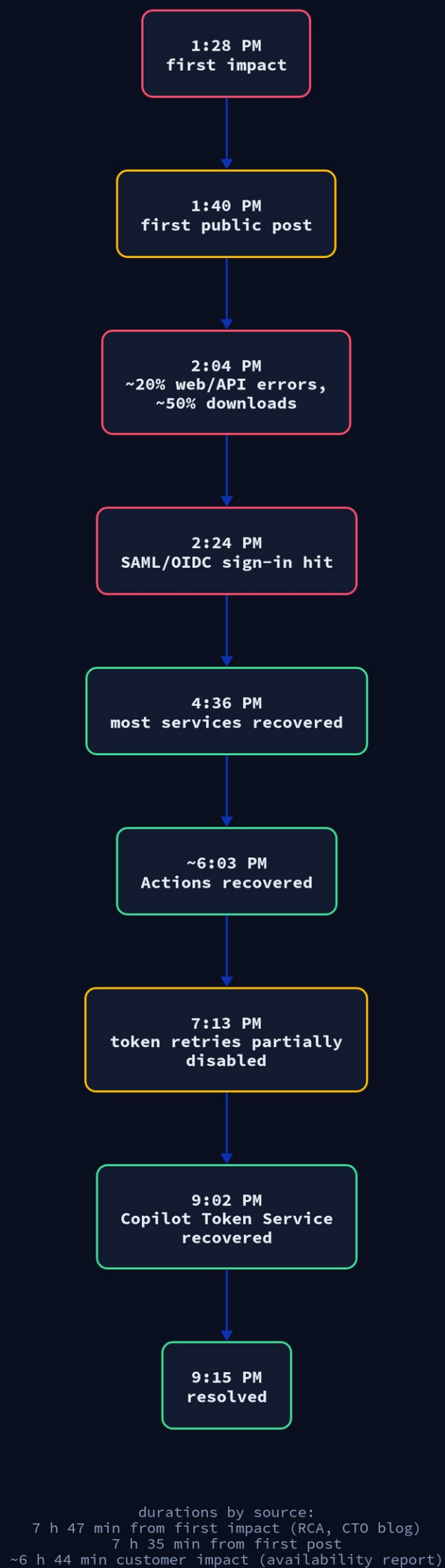


the fix

retry at ONE layer + a retry budget (e.g. $\leq 10\%$ of traffic, illustrative)
-> worst case = that layer's attempts, not a product

Timeline, Aug 17, 2026 (UTC, 12-hour)

src/timeline_aug17.d2 · svg/timeline_aug17.svg · png/timeline_aug17.png



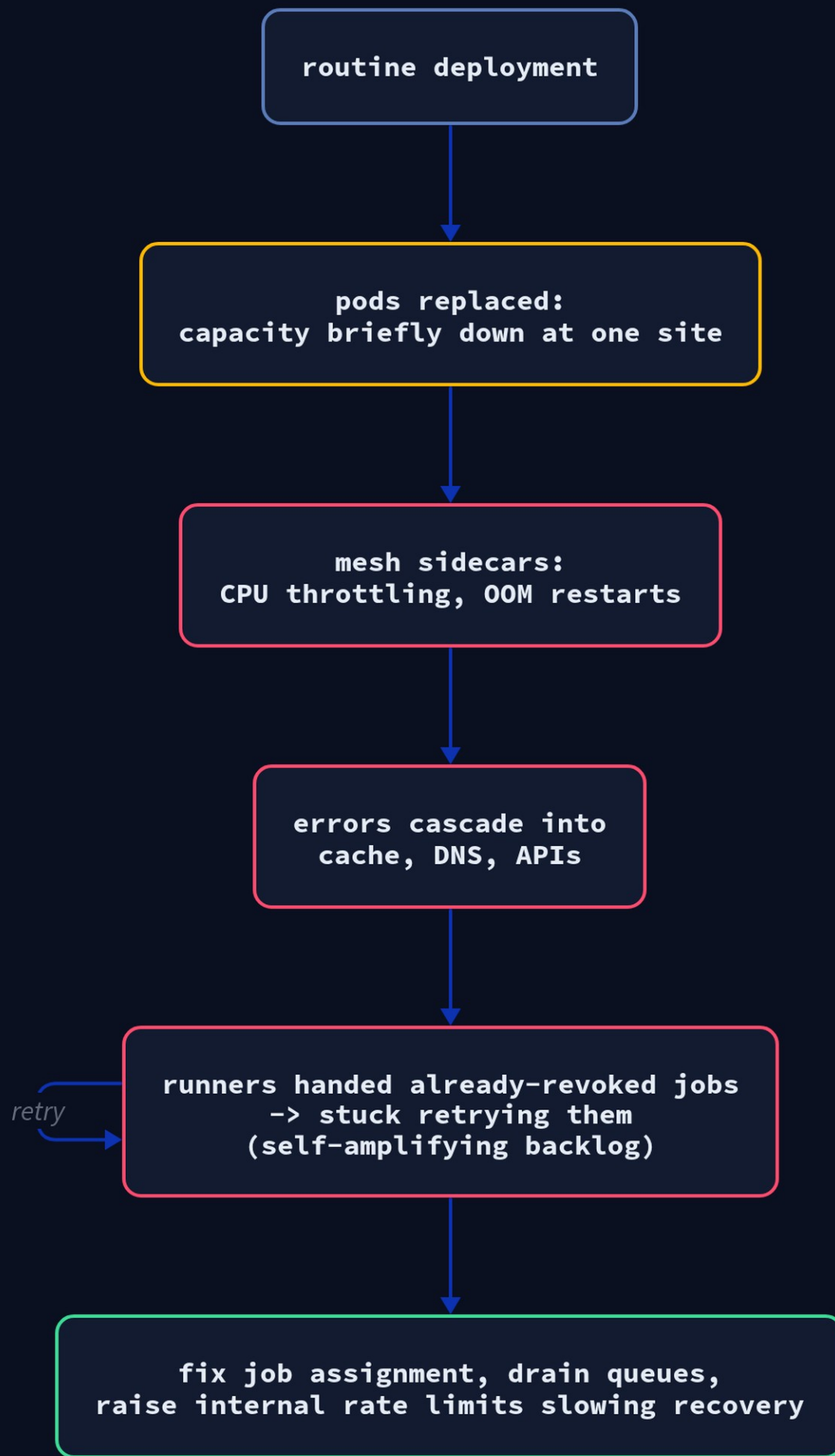
Failure dynamics: first event, signal, shared resource, containment, trade-off, recovery

[src/failure_dynamics.d2](#) · [svg/failure_dynamics.svg](#) · [png/failure_dynamics.png](#)



Aug 6, 2026: the self-amplifying Actions backlog

src/aug06_backlog.d2 · svg/aug06_backlog.svg · png/aug06_backlog.png



same shape as Aug 17:
a mesh sidecar without headroom,
then retries adding load faster
than the system could clear it

Monday: six recommendations (our opinion), each with what it does NOT solve

[src/monday_policy.d2](#) · [svg/monday_policy.svg](#) · [png/monday_policy.png](#)

change

what it does NOT solve

1 • retry at one layer, not every layer
(edge or client owns retries; hops in between fail fast)

clients you don't ship (an editor extension
on laptops you'll never touch)

2 • retry budget + backoff with jitter
(GitHub CTO: retry limits, retry budgets, variable timeouts)

adds no capacity: if first attempts alone are too many,
it only stops you making it worse

3 • defend the server from clients you can't fix
(per-client limits, edge shedding, 403 / 429 + Retry-After)

you're choosing who fails; a buggy client
may ignore the signal anyway

4 • autoscale on the limit you actually hit
(sidecar concurrency, not just the host); alert on saturation

a spike faster than new capacity can start:
that still needs headroom

5 • shrink the shared path
(isolate critical systems, remove shared dependencies)

isolation limits how far one failure spreads,
not whether the first one happens

6 • rehearse the ramp
(site by site; healthy telemetry for N minutes per step)

slower by design: users wait longer.
Lowers, doesn't remove, second-wave risk:
clients stuck in a retry loop must be blocked first